CoLLM: Industrial Large–Small Model Collaboration With Fuzzy Decision-Making Agent and Self-Reflection

Haiteng Wang, *Member, IEEE*, Lei Ren, *Senior Member, IEEE*, Tuo Zhao, and Lu Jiao

***Abstract*—In industrial applications, large models have exhibited superior generalization capabilities that are unattainable with smaller models. However, when faced with edge scenarios and highly diverse industrial samples, their deployment remains challenging due to high computational costs and unreliable output. To address these challenges, we propose a large–small model collaborative framework (CoLLM), which dynamically selects between small and large models based on the characteristics exhibited by the samples. Specifically, this approach estimates uncertainty from input samples to guide model selection: low-uncertainty samples are processed by the small model for efficiency, while high-uncertainty or complex samples are routed to the large model for improved accuracy. It first constructs a fuzzy decision-making agent based on the fuzzy neural network to assess sample complexity and determine the appropriate model for inference. Furthermore, a self-reflection mechanism is proposed to refine the large model's output, reducing the risk of unreliable output. Experimental results in industrial time-series datasets demonstrate that our framework improves the computational efficiency of large models up to 14.54× while maintaining or improving prediction accuracy.**

***Index Terms*—Foundation model, industrial large model, large language model (LLM), industrial time series, mixture of experts (MoE).**

I. Introduction

DATA-DRIVEN industrial intelligence has enabled automated anomaly detection, task planning, and predictive maintenance without relying on expert knowledge [1], [2]. In earlier years, small-scale deep learning models proved effective but were often limited to single-task applications. In contrast, recent advances in large models (LMs), particularly large language models (LLMs), have demonstrated transformative capabilities [3], [4], [33]. These models exhibit strong generalization, allowing them to adapt rapidly to diverse industrial data patterns and deliver higher accuracy in complex, multitask settings.

Despite their impressive performance, LM still face two major challenges in practical industrial applications: their massive parameter sizes (e.g., 200B for GPT-4o [5] and 671 B for DeepSeek-V3 [6]) and autoregressive nature, resulting in extremely high inference costs [7]. In addition, LMs face huge uncertainty in processing certain industrial data, such as sensor signals affected by noise, drift, or missing values [8], [9], [10], [11], leading to unreliable responses.

Recently, researchers have explored collaborative inference strategies [12] that take the advantage of small model (SM) and LM. SM offer lower computational costs, faster inference speeds, and stable decision-making accuracy, whereas LMs provide superior generalization and the capability to handle complex tasks. Collaborative inference employs SM for rapid processing while invoking LMs only when necessary for complex decision making, thereby reducing computational costs while maintaining accuracy. Based on this, some studies adopt task-level collaborative strategies [13], [14], [15] to allocate queries to expert models with different specializations. For instance, composition of experts (CoE) approach [13] uses a router model to dynamically assign each input prompt to the most relevant expert. However, such methods typically require well-defined task boundaries. In addition, even within the same task category, the complexity of different input samples can vary significantly. The static task-based division cannot dynamically adapt to the complexity of the sample. To overcome these limitations, recent research has proposed sample-level collaborative methods [7], [16], [17], [18], [19] that adaptively invoke LM or SM based on sample complexity to optimize computational efficiency. For example, RouteLLM [17] balances cost and response quality by dynamically selecting between powerful and cost-effective models. This approach can reduce LM usage by 50% while maintaining response quality. However, to develop a fast and reliable large–small model collaborative inference framework, the following technology issues need to be addressed.

First, existing collaborative inference methods mainly target natural language processing (NLP) tasks, where difficulty assessment is based on discrete semantic tokens (e.g., semantic similarity and topic distribution). However, industrial applications primarily involve continuous time-series data (e.g., sensor

Received 30 March 2025; revised 12 June 2025; accepted 24 July 2025. Date of publication 31 July 2025; date of current version 2 April 2026. This work was supported in part by the National Key Research and Development Program of China Grant 2023YFB3308000 and in part by the National Science Foundation of China (NSFC) under Grant 62225302, Grant 623B2014, and Grant 62173023. Recommended by Associate Editor H. Yu. *(Corresponding author: Lei Ren.)*

Haiteng Wang, Tuo Zhao, and Lu Jiao are with the School of Automation Science and Electrical Engineering, Beihang University, Beijing 100191, China (e-mail: wanghaiteng@buaa.edu.cn; zhaotuo@buaa.edu.cn; zy2403804@buaa.edu.cn).

Lei Ren is with the School of Automation Science and Electrical Engineering, Beihang University, Beijing 100191, China, also with Hangzhou International Innovation Institute, Beihang University, Hangzhou 311115, China, and also with the State Key Laboratory of Intelligent Manufacturing System Technology, Beijing 100854, China (e-mail: renlei@buaa.edu.cn).

Digital Object Identifier 10.1109/TFUZZ.2025.3594229

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

signals, process parameter curves), which exhibit dynamic variability and ambiguous boundaries [8], [9], [10], [11]. Existing methods struggle to assess the complexity of such data effectively, leading to inefficient resource allocation between SM and LM.

Second, most approaches implicitly assume that LMs consistently outperform SM for all samples within a given task. However, variability in sample characteristics can cause performance differences between SM and LM. For instance, early-stage degradation detection often depends on capturing small fluctuations in sensor data or subtle details in vibration signals. SM may excel at capturing such local features, while larger models, due to overparameterization, risk catastrophic errors. We emphasize that LM and SM have distinct strengths depending on sample characteristics, and the appropriate model should be selected accordingly. Existing frameworks lack error correction mechanisms at the individual sample level, posing significant risks to industrial decision-making processes.

To address these challenges, we propose a fuzzy large–small model collaborative framework (CoLLM) to reduce computational cost while improving reliability for LMs. The basic idea is that industrial samples exhibit varying characteristics, making some better handled by SM and others by large ones. Unlike static task-level collaboration, our approach dynamically selects the appropriate LM or SM based on individual sample characteristics. Specifically, we propose a fuzzy decision-making agent that quantifies sample uncertainty based on a fuzzy neural network (FNN), identifying whether LM inference is required. To enhance reliability in LMs, we propose a self-reflection mechanism that assesses the LM's prediction confidence. When confidence is low, SM are employed for correction. The main contributions are as follows.

A sample-level fuzzy large–small model collaborative framework is proposed to dynamically select between LM and SM based on individual sample characteristics. This fine-grained decision-making strategy significantly reduces computational cost while maintaining or improving prediction reliability.

A fuzzy decision-making agent based on an FNN is designed to quantify sample uncertainty and enable autonomous decisions on whether to invoke the LM.

A self-reflection mechanism is proposed to refine the prediction of the LM. When the uncertainty in the LM's output is high, an SM is invoked to refine the output, mitigating unreliable outputs in industrial scenarios.

II. Related Work

A. LMs for Time-Series Analysis

Current time-series modeling approaches are mainly classified into two categories: pretrained large time-series models [20], [21] and fine-tuning LLMs [22], [23], [24], [34] for time-series tasks. Pretrained large time-series models encompass Transformer-based models, contrastive learning-based models, convolutional network-based models, decomposition-based models, graph neural network-based models, and generative models. For example, the Transformer-based model Moirai [20] addresses heterogeneity in arbitrary time-series data by leveraging attention mechanisms for different variables and mixed parametric distributions. Similarly, the Timer model [21] unifies heterogeneous time-series data into a single-sequence structure (S3) and reformulates time-series forecasting, imputation, and anomaly detection as a unified generative task, thereby mitigating data heterogeneity issues. However, these models require extensive time-series data for effective performance.

Another alternative strategy is to fine-tune LLMs for time-series tasks. For instance, TIME-LLM [22] fine-tunes LLMs using patch reprogramming and employs patch as prompt to address modality heterogeneity, enhancing the model's ability to understand time-series data. The aLLM4TS model [23] adopts a two-stage pretraining strategy combined with multipatch prediction fine-tuning, improving LLM adaptability and optimizing patch-based time-series learning. In addition, the S2IP-LLM model [24] introduces a specialized tokenization module and utilizes semantic anchors for context prompting, effectively optimizing time-series data embeddings and enhancing representation learning. In addition, the studies [8], [9], [10], [11] propose time-series modeling approaches based on fuzzy learning to capture the inherent uncertainty and complexity of time-series data, providing novel insights and methodologies for time-series forecasting.

However, these methods inherently incur substantial inference costs, making real-time inference and deployment on resource-constrained industrial systems challenging [35], [36], [37]. This study introduces a FNN that generates confidence scores based on input samples to determine whether to trigger LM inference.

B. Collaborative Inference Between SM and LM

The collaborative inference methods between LM and SM balance computational efficiency and decision accuracy by leveraging the strengths of both. Task-level collaborative methods [13], [14], [15] use task type as the scheduling unit. A typical example is the CoE framework [13], which incorporates an intent recognition mechanism to assign tasks to the most suitable expert model or to enable multiple models to collaborate. While such methods achieve coarse-grained task allocation, they do not fully account for the complexity differences within individual samples. As a result, simple samples may unnecessarily consume LM resources, while complex samples risk misallocation, leading to decision biases. To overcome these limitations, recent studies have proposed sample-level collaborative methods [7], [16], [17], [18], [19] that adaptively invoke LM or SM by dynamically assessing the complexity of individual samples to optimize computational efficiency. For instance, FrugalGPT [7] reduces LLM usage costs through prompt adaptation, LLM approximation, and LLM cascading, learning to allocate queries efficiently. Similarly, RouteLLM [17] balances cost and response quality by dynamically selecting between powerful and cost-effective models, cutting LLM invocations by 50% while maintaining response quality.

However, existing methods mainly focus on natural language processing tasks, where routing decisions rely on semantic features, such as syntactic complexity and topic consistency. In contrast, industrial applications primarily involve sensor time-series

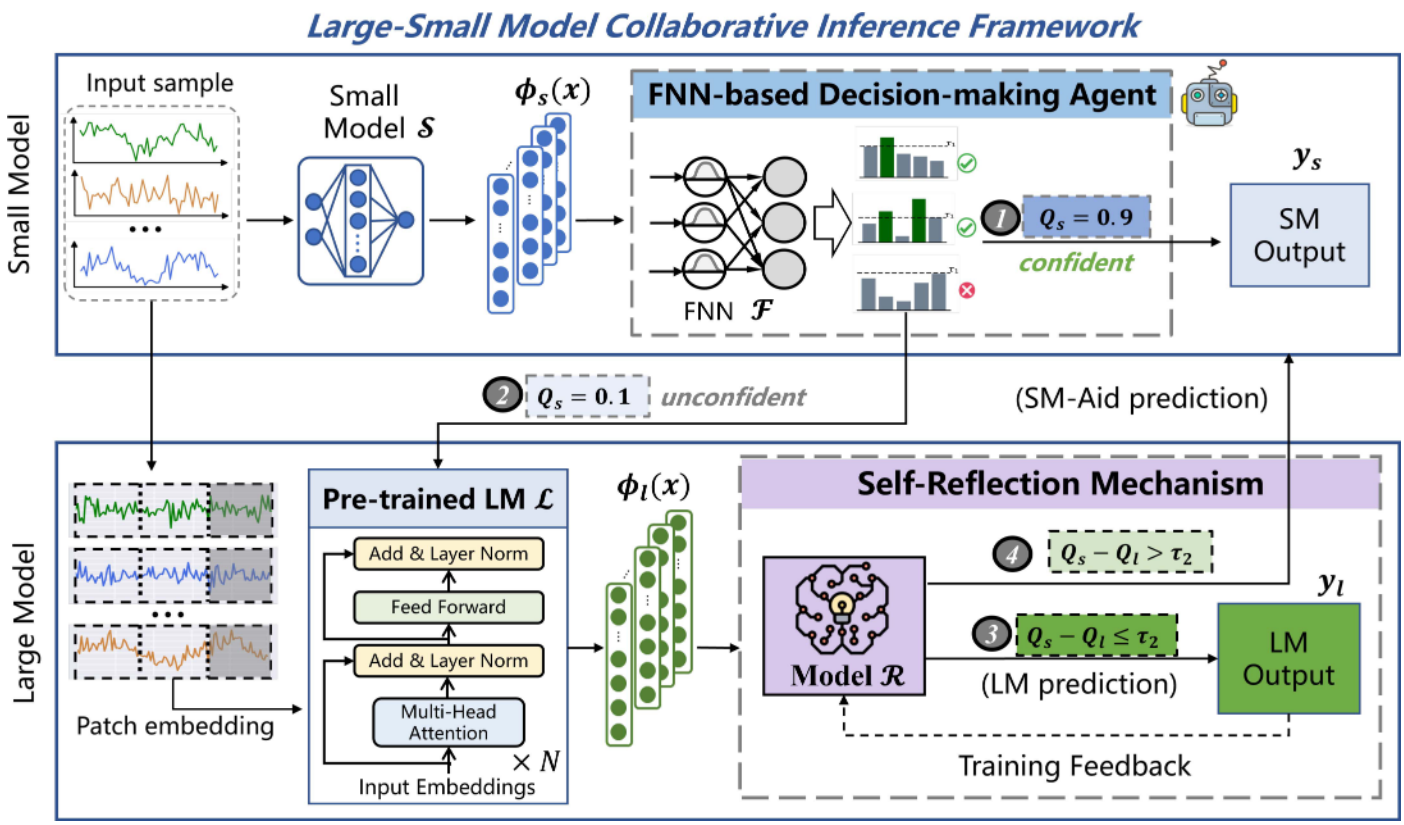


Fig. 1. Fuzzy large–small model collaborative framework. This framework employs a cascade inference strategy, where the SM $\mathcal{S}$ first performs rapid inference, and the confidence score Q_s is computed using an FNN $\mathcal{F}$ to assess prediction reliability. If $Q_s \geq \tau_1$, then the SM's prediction $\mathbf{y}_s$ is directly output. Otherwise, the LM $\mathcal{L}$ is invoked, and its confidence score Q_l is computed for self-reflection. If the confidence difference $\Delta = Q_s - Q_l > \tau_2$, the predictions from both models are fused as $\mathbf{y}_{\text{final}} = \mathcal{G}(\mathbf{y}_s, \mathbf{y}_l; \Delta)$. Otherwise, the LM's prediction $\mathbf{y}_l$ is the output.

data (e.g., vibration spectra and process parameter sequences), which exhibit strong temporal correlations, high noise interference, and ambiguous pattern boundaries. Traditional semantic-based evaluation mechanisms struggle to effectively quantify the processing difficulty of such data.

III. Methods

A. Fuzzy Large–Small Model Collaborative Framework

The fuzzy large–small model collaborative framework employs a cascade inference strategy, adaptively selecting either a LM or SM for inference to balance computational efficiency with predictive accuracy. The core process consists of the following three stages.

1) Fast Inference by SM: In industrial tasks, input data x are sampled from the sample space $\mathcal{X} \subset \mathbb{R}^{t\times d}$. The inference process begins with a lightweight, SM $\mathcal{S}$ for rapid prediction. The SM is designed to provide real-time responses with significantly lower computational complexity compared to the LM. This stage filters simple samples via initial predictions from the SM, reducing the frequency of LM invocation.

Given the input space $\mathcal{X}$ and output space $\mathcal{Y}$, the SM with parameters $\boldsymbol{\theta}_s$ defines the mapping

$$\mathcal{S} : \mathcal{X} \to \mathcal{Y}, \quad \mathbf{y}_s = \mathcal{S}(x; \boldsymbol{\theta}_s) \tag{1}$$

where $\mathbf{y}_s$ represents the predicted value in regression tasks.

2) Decision-Making Agent Based on FNN: To determine whether to invoke the LM, a fuzzy decision agent $\mathcal{F}$ is constructed based on an FNN. This agent generates a confidence score $Q_s \in [0, 1]$ based on the input sample. If Q_s is below a predefined threshold τ_1, then the SM's prediction is deemed unreliable, and the LM is invoked for inference. Otherwise, $\mathbf{y}_s$ is directly accepted. This mechanism leverages an FNN to handle the uncertainty of the industrial sample.

The FNN takes processed features $\phi_s(x)$ output from the SM, where $\phi_s : \mathcal{X} \to \mathbb{R}^{t\times d_s}$ represents the feature extractor of the SM. With parameters $\boldsymbol{\theta}_f$, the confidence score is computed as

$$\mathcal{F} : \mathbb{R}^{t\times d_s} \to [0, 1], \quad Q_s = \mathcal{F}(\phi_s(x); \boldsymbol{\theta}_f). \tag{2}$$

The decision function $\mathcal{D}_1(Q_s)$ is defined as

$$\mathcal{D}_1(Q_s) = \begin{cases} 1, & Q_s \geq \tau_1 \Longrightarrow \text{Accept } \mathbf{y}_s \\ 0, & Q_s < \tau_1 \Longrightarrow \text{Activate LM } \mathcal{L}. \end{cases} \tag{3}$$

3) LM Deep Inference and Self-Reflection: If the SM's prediction is deemed unreliable (i.e., $\mathcal{D}_1(Q_s) = 0$), then the LM $\mathcal{L}$ is invoked to conduct deep inference, generating a fine-grained prediction $\mathbf{y}_l$

$$\mathbf{y}_l = \mathcal{L}(x; \boldsymbol{\theta}_l). \tag{4}$$

In addition, to mitigate unreliable predictions from the LM, a self-reflection mechanism $\mathcal{R}$ is introduced. This mechanism quantifies potential predictive biases within the feature space of the LM and compares them with the SM. Specifically, given an input $x \in \mathcal{X}$, the LM's feature extractor $\phi_l : \mathcal{X} \to \mathbb{R}^{t \times d_l}$ generates high-dimensional representations. The self-reflection model is defined as

$$Q_l = \mathcal{R}(\phi_l(x); \boldsymbol{\theta}_r) \tag{5}$$

where $\boldsymbol{\theta}_r$ are the parameters of the self-reflection model, and the output $Q_l \in [0, 1]$ represents the confidence score of the LM's prediction. The confidence difference $\Delta = Q_s - Q_l$ is then compared against a threshold τ_2. If $\Delta > \tau_2$, then it indicates that the LM's confidence is significantly lower than that of the SM, and thus, the SM-aided inference is activated. The self-reflection mechanism is formulated as

$$\mathcal{D}_2(\Delta) = \begin{cases} 1, & \Delta \leq \tau_2 \Longrightarrow \text{Accept } \mathbf{y}_l \\ 0, & \Delta > \tau_2 \Longrightarrow \text{SM-aid prediction.} \end{cases} \tag{6}$$

The SM-aided prediction function $\mathcal{G}$ is given by

$$\mathbf{y}_{\text{final}} = \mathcal{G}(\mathbf{y}_s, \mathbf{y}_l; \Delta). \tag{7}$$

As a practical implementation, this study adopts a straightforward ensemble strategy, combining the outputs of the SM and LM such that the final prediction is given by $\mathcal{G}(\cdot) = \frac{\mathbf{y}_s + \mathbf{y}_l}{2}$. This simple yet effective ensemble approach leverages the complementary strengths of both models to enhance robustness. In addition, the framework is extensible to more sophisticated fusion strategies.

4) Inference Pipeline: Overall, this study presents a fuzzy large–small model collaborative framework tailored for industrial applications, enhancing inference speed and reliability. The framework first utilizes a lightweight SM for rapid feature extraction and confidence evaluation. For high-confidence simple samples, predictions are directly output to reduce computational costs. When high uncertainty is detected, the LM is invoked. If the LM exhibits high reliability, then its predictions are adopted; otherwise, a SM-aided prediction is employed to generate the final prediction.

B. Fuzzy Decision-Making Agent

The fuzzy decision-making agent primarily utilizes an FNN to generate a confidence score for the SM. This section provides a detailed description of the implementation of the FNN architecture and the strategy for generating confidence.

1) FNN Architecture: In industrial settings, time-series data often contain significant uncertainty, making it challenging for deterministic deep learning methods to manage noise and strong dynamics. This issue is particularly pressing in predictive maintenance, where accurately evaluating model confidence is critical for downstream decisions. Due to continuous and evolving time-series inputs, current methods often fail to capture complex patterns, limiting their ability to schedule tasks appropriately between LM and SM.

Algorithm 1: Fuzzy Large–Small Model Collaborative Inference Algorithm.

Input: Industrial input sample $x \subset \mathbb{R}^{t \times d}$
Output: Final prediction result $\mathbf{y}_{\text{final}} \subset \mathbb{R}^1$

Extract features by the small model: $\phi_s(x) \in \mathbb{R}^{t \times d_s}$;
Compute the confidence score via the decision agent: $Q_s \leftarrow \mathcal{F}(\phi_s(x); \boldsymbol{\theta}_f)$;
if $Q_s \geq \tau_1$ **then**
  `// Accept small model's prediction`
  return $\mathbf{y}_s \leftarrow \mathcal{S}(x; \boldsymbol{\theta}_s)$
else
  Extract features by the large model: $\phi_l(x) \in \mathbb{R}^{t \times d_l}$;
  Compute the confidence score of the large model: $Q_l \leftarrow \mathcal{R}(\phi_l(x); \boldsymbol{\theta}_r)$;
  Compute the confidence difference: $\Delta \leftarrow Q_s - Q_l$;
  if $\Delta \leq \tau_2$ **then**
    `// Accept large model's prediction`
    return $\mathbf{y}_l \leftarrow \mathcal{L}(x; \boldsymbol{\theta}_l)$
  else
    `// Refine the prediction with the small model`
    $\mathbf{y}_{\text{final}} \leftarrow \mathcal{G}(\mathbf{y}_s, \mathbf{y}_l; \Delta)$;
    return $\mathbf{y}_{\text{final}}$
  end
end

To address these issues, this article integrates fuzzy learning methods into the deep learning framework to facilitate confidence estimation for industrial large–small model collaboration prediction. By leveraging learnable fuzzy membership functions, fuzzy learning transforms uncertain and noisy time-series features into fuzzy metrics that capture uncertainty, thereby providing robust support for confidence prediction. Compared with traditional deterministic neural networks, FNN offer greater resilience to noise and heterogeneity in industrial data, enabling more reliable decision making under uncertain conditions.

First, we decompose the feature representation $\phi_s(x)$ produced by the SM, handling each dimension z_t^d independently. For each feature dimension d, we define an individual fuzzy membership function that maps the data in this dimension to the [0,1] fuzzy membership range. Specifically, we adopt a Gaussian membership function, as shown in the following:

$$f_d(z_t^d) = \mathbf{F}(z_t^d, \mu_d, \sigma_d) = e^{-(z_t^d - \mu_d)^2 / \sigma_d^2}. \tag{8}$$

Note that the fuzzy membership function for each dimension is not manually determined by expert knowledge. Instead, we set the fuzzy mean μ and fuzzy variance σ^2 as learnable parameters in the neural network so that a suitable Gaussian membership function can be learned for each dimension during training. The fuzzy mean μ captures the central trend of the sensor data along each dimension, reflecting the typical or expected value of the corresponding sensor during operation. The fuzzy variance σ^2 characterizes the degree of dispersion around the fuzzy mean,

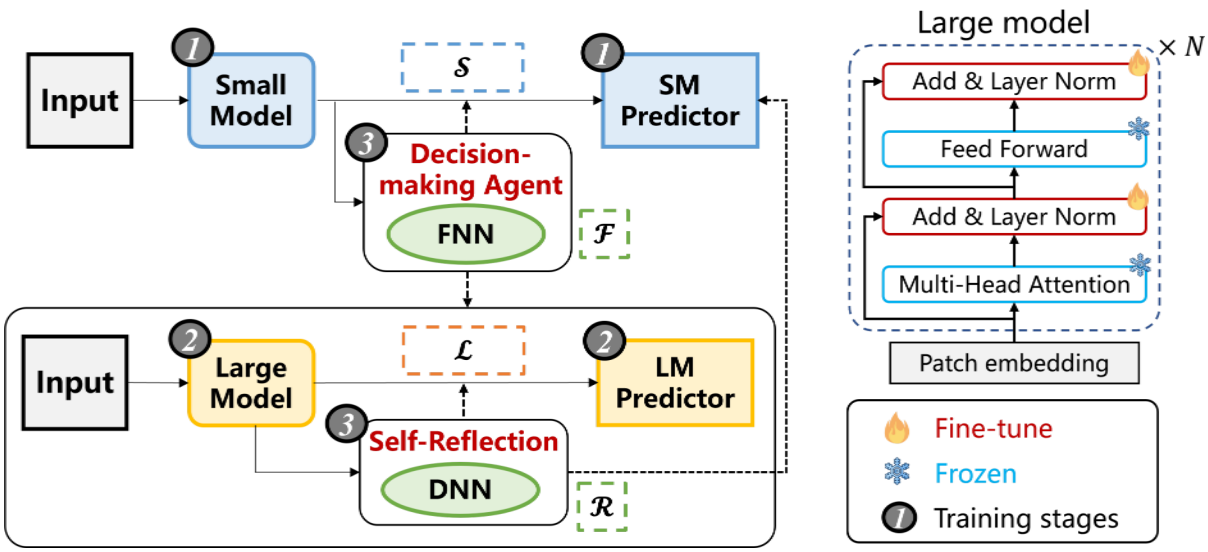


Fig. 2. Training process of CoLLM. The framework employs a three-stage progressive training strategy with hierarchical parameter freezing to reduce gradient interference. The numbers (1, 2, 3) and the corresponding blue, yellow, and green blocks indicate the training stages. In the LM, the multihead attention and feed-forward layers remain frozen throughout the training process.

effectively quantifying the uncertainty inherent in the sensor readings.

Next, the membership values $f_d(z_t^d)$ for all dimensions are merged into a fuzzy feature matrix $\mathcal{M}$. The final expression of the fuzzy feature matrix $\mathcal{M}$ is given by

$$\mathcal{M} = \begin{bmatrix} f_1(x_1^1) & f_2(x_1^2) & \cdots & f_d(x_1^d) \\ f_1(x_2^1) & f_2(x_2^2) & \cdots & f_d(x_2^d) \\ \vdots & \vdots & \ddots & \vdots \\ f_1(x_T^1) & f_2(x_T^2) & \cdots & f_d(x_T^d) \end{bmatrix}. \tag{9}$$

These fuzzy features extracted by the FNN effectively capture the uncertainty present in the input data, thereby enhancing the model's predictive performance in complex time-series prediction tasks. Moreover, they also provide crucial information for subsequent confidence estimation, further improving the overall prediction accuracy.

2) Confidence Score Strategy: In domains such as natural language processing, confidence scores commonly rely on metrics derived from the softmax output, such as the maximum output value or the entropy of the output distribution, to assess confidence. However, in regression tasks, model outputs are no longer probability distributions but continuous or multidimensional real-valued vectors, rendering conventional maximum-probability-based confidence estimation methods inapplicable.

To determine whether a LM should be used, we adopt a confidence-based decision strategy that adaptively learns confidence scores for SM, as illustrated in Fig. 1. When the confidence score from the SM exceeds its designated threshold, the prediction is deemed reliable; otherwise, the LM is activated to perform further predictions.

Given the predicted results $\mathbf{y}_s$ and the true label $\mathbf{y}_s^*$, we then derive a confidence label Q_s^* as follows:

$$Q_s^* = 1 - \tanh\left(\frac{|\mathbf{y}_s - \mathbf{y}_s^*|}{\alpha}\right. . \tag{10}$$

Here, tanh is a nonlinear function that confines the value of Q_s^* to the range [0,1], and α is a parameter that governs the sensitivity of the function to prediction errors.

The process of predicting the confidence score Q_s can be expressed as

$$Q_s = \mathcal{F}(\phi_s(x)) = \sigma(W\phi_s(x) + b) \tag{11}$$

where σ is the sigmoid function, W is the weight vector, and b is the bias term. We employ the mean squared error as the loss function for the confidence predictor, aiming to minimize the discrepancy between the predicted confidence score Q_s and the confidence label Q_s^*.

During inference, the confidence thresholds for each subnetwork are predefined and remain fixed throughout the process. Specifically, when sufficient computational resources are available, higher thresholds are assigned to the SM, allowing more samples to propagate to LM for improved prediction accuracy. Conversely, under constrained conditions, lowering the thresholds for SM reduces computational overhead and accelerates inference, thereby enabling the model to adapt to limited resources while still maintaining acceptable predictive performance.

C. Self-Reflection Mechanism

The self-reflection model $\mathcal{R}$ is implemented using a fully connected network (FCN). Its input is the latent time-series feature matrix $\phi_l(x) \in \mathbb{R}^{t \times d_l}$ extracted by the LM. To reduce the complexity of high-dimensional time-series features, the input is first flattened along the temporal dimension, transforming it into

a static 1-D feature vector. Then, a single-layer fully connected projection is designed to map features to a confidence scalar Q_l, formulated as

$$Q_l = \mathcal{R}(\phi_l(x)). \tag{12}$$

The FCN is trained using a supervision signal based on the prediction error of the LM. Similarly, the target confidence Q_l^* can be defined as

$$Q_l^* = 1 - \tanh\left(\frac{|\mathbf{y}_l - \mathbf{y}_l^*|}{\alpha}\right) \tag{13}$$

where $\mathbf{y}_l^*$ represents the ground-truth value, $\mathbf{y}_l$ is the original prediction of the LM $\mathcal{L}$, and $\alpha > 0$ is the residual scaling coefficient controlling error sensitivity. The training objective is formulated as follows:

$$\mathcal{L}_{\text{loss}} = \frac{1}{N}\sum_{i=1}^{N}(Q_{l_i} - Q_{l_i}^*)^2 \tag{14}$$

where N denotes the batch size, and Q_{l_i} and $Q_{l_i}^*$ represent the predicted and target confidence scores for the ith sample, respectively.

During inference, the self-reflection model $\mathcal{R}$ quantifies the potential prediction bias based on the feature representations extracted by ϕ_l, producing a confidence score $Q_l \in [0, 1]$. The confidence difference $\Delta = Q_s - Q_l$ between the LM confidence Q_l and the SM confidence Q_s is computed and compared with a predefined threshold τ_2. If $\Delta \leq \tau_2$, then the LM prediction $\mathbf{y}_l$ is accepted; otherwise, the SM assistive mechanism is triggered to prevent unreliable predictions.

D. LM Architecture and Training Optimization

1) LM Architecture: The proposed LM is based on the pretrained GPT-2 architecture, employing a frozen parameter strategy that retains the self-attention mechanism and feed-forward neural network modules learned during pretraining. These components capture rich language features and are leveraged for time-series processing. To accommodate time-series data, the input layer is redesigned with a patch embedding mechanism. Specifically, time-series data are segmented into multiple local patches, each mapped to high-dimensional vectors via a linear transformation, forming input embeddings.

2) Multistage Training Strategy: A three-stage progressive training strategy is proposed to achieve modular cooperative inference and parameter decoupling optimization, employing a hierarchical parameter freezing mechanism to reduce gradient interference risks.

Training Stage 1: The SM ($\mathcal{S}$) is first trained independently, with all other modules frozen. End-to-end optimization minimizes prediction errors, enabling $\mathcal{S}$ to establish time-series feature representations.

Training Stage 2: The LM ($\mathcal{L}$) is fine-tuned with $\mathcal{S}$'s parameters frozen to prevent optimization interference from $\mathcal{S}$'s backpropagation path.

Training Stage 3: In the final stage, the self-reflection model ($\mathcal{R}$) and decision agent ($\mathcal{F}$) are trained, with all other module parameters frozen. $\mathcal{R}$ and $\mathcal{F}$ learn the joint distribution of input features and predictions via neural networks, constructing a confidence-driven dynamic routing mechanism.

IV. Experiment

This section describes the experimental setup and analysis. All experiments are conducted on a workstation with an Intel Xeon Silver 4210 CPU at 2.20 GHz and four NVIDIA GeForce RTX 3090 GPU.

A. Experimental Setup

1) Dataset: In this section, we will introduce the aero-engine datasets used in the article. The Commercial Modular Aero-Propulsion System Simulation (CMAPSS) dataset [25], released by the National Aeronautics and Space Administration in 2008, has become a widely used benchmark in the field of remaining useful life (RUL) prediction. This dataset provides a detailed record of the complete degradation process of turbofan engines from normal operation to failure, offering researchers a wealth of realistic degradation data.

Within the CMAPSS dataset, the operational status of the turbofan engine is monitored by 21 different sensors, measuring parameters, such as time, pressure, and rotational speed. However, since the readings of sensors 1, 5, 6, 10, 16, 18, and 19 remain constant throughout the process, they show little correlation with changes in RUL. Consequently, these sensors are removed, and the remaining 14 sensors are used for prediction tasks. In addition, the CMAPSS dataset is divided into multiple subsets to accommodate testing under different scenarios and operating conditions. In this study, we selected two of these subsets, FD001 and FD003, to evaluate the performance of our model.

2) Data Process: First, the entire dataset is normalized to minimize the impact of distribution mismatches between the training and testing data

$$x_f = \frac{x_f - u_f}{\sigma_f} \tag{15}$$

where x_f represents the feature data, while u_f and σ_f, respectively, denote the mean and standard deviation computed over the entire dataset.

Following previous works [26], we employ a sliding-window approach to process the normalized dataset and segment the entire input sequence. Specifically, the window size is set to 50 with a stride of 1.

3) Hyperparameters: During training, 20% of the original dataset is randomly selected as the validation set. The backbone model is optimized using the Adam optimizer. Training is conducted for 100 epochs with an early stopping mechanism. Detailed hyperparameter settings are provided in Table I.

4) Evaluation Metrics: We evaluate the proposed model using two commonly employed metrics in RUL prediction: the root-mean-squared error (RMSE) and the mean absolute error (MAE)

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum\nolimits_{i=1}^{N}(\hat{y}_i - y_i)^2} \tag{16}$$

TABLE I
DEFAULT HYPERPARAMETERS OF DYNAMIC FRAMEWORK

Parameters	FD001	FD003
Batch size	256	256
Learning rate	2e−3	2e−3
Window size t	50	50
Input size d	14	14
$\mathcal{S}$ Embedding size	32	32
$\mathcal{S}$ hidden size	64	64
Feature size d_s	32	32
Number of f_d function	64	64
$\mathcal{L}$ Patch stride	4	4
$\mathcal{L}$ Patch size	4	4
$\mathcal{L}$ Embedding size	768	768
Feature size d_l	768	768

$$\text{MAE} = \frac{1}{N}\sum_{i=1}^{N}|\hat{y}_i - y_i| \tag{17}$$

where N is the number of samples, and $\hat{y}_i$ and y_i are the true label and the output of the ith sample, respectively.

5) Flops Analysis: Floating-point operations (FLOPs) refer to the number of FLOPs performed by the model in a single computation. This metric is used to measure the computational complexity of the model. FLOPs are independent of the runtime environment [whether CPU, GPU, or tensor processing unit (TPU)], reflecting only the model's inherent computational demands. Generally, the larger the FLOPs, the higher the computational overhead. Therefore, under the same level of accuracy, a model with fewer FLOPs is more efficient and better suited for industrial applications.

B. Main Results

We conducted experiments on the FD001 and FD003 datasets using LMs, such as GPT-2, Llama2-7b, One Fits All, Autotimes, Qwen2.5-0.5b, and Qwen3-0.6b. To ensure fairness, the subnetworks in the dynamic architectures employ the same parameter configurations as those of their base networks. Models with 1-Epoch training refer to training for a single epoch, primarily to enable the language model to acquire basic capability in handling time-series data, while fine-tuning indicates thorough fine-tuning with multiple epochs. We then implemented CoLLM by leveraging the fine-tuned Transformer encoder as the SM and One Fits All as the LM. We varied the confidence thresholds and evaluated the corresponding performance and speed of CoLLM. A higher threshold τ_1 means that fewer samples exit at the SM, thus requiring more computational resources for the LM's prediction. τ_2 represents the threshold for LLMs to engage in self-reflection. A higher value of τ_2 corresponds to fewer samples being subjected to the model's self-reflection mechanism. For the FD001 dataset, the threshold settings are as follows: 1) CoLLM-A [0.3, 0.1], 2) CoLLM-B [0.4, 0.1], and 3) CoLLM-C [0.6, 0.05]. For the FD003 dataset, the thresholds are configured as: 1) CoLLM-A [0.15, 0.1], 2) CoLLM-B [0.4, 0.1], and 3) CoLLM-C [0.6, 0.05]. The speedup is measured by calculating FLOPs and comparing it with that of the baseline LM. The experimental results are given in Table II.

On the FD001 dataset, CoLLM-A achieves an RMSE of 12.45 and an MAE of 9.13, with accuracies of 99.1% and 97.3%, respectively, while reducing FLOPs by a factor of 3.88. CoLLM-C achieves an RMSE of 12.33 and an MAE of 8.86, with accuracies of 100.1% and 100.3%, respectively, while reducing FLOPs by a factor of 1.26. On the FD003 dataset, CoLLM-A achieves an RMSE of 11.26 and an MAE of 7.42, with accuracies of 99.3% and 95.2%, respectively, while reducing FLOPs by a factor of 14.54. CoLLM-C achieves an RMSE of 11.11 and an MAE of 7.12, with accuracies of 101.44% and 100.42% respectively, while reducing FLOPs by a factor of 1.57. These experimental results show that, despite performance and dataset differences, CoLLM can accelerate computations (up to 14.54×) while sustaining minimal accuracy loss or even achieving better performance. Fig. 3 presents examples of several prediction results, showing that the predicted RUL curves closely match the ground truth. From the lower part of Fig. 3, one can observe that the prediction error aligns well with the actual error, demonstrating the effectiveness of our confidence prediction strategy.

C. Effect of the Self-Reflection

To assess the effectiveness of the self-reflection mechanism, we conducted an ablation study. We tested the CoLLM on the FD003 dataset and adjusted various confidence thresholds for the submodels. The term "number of sample reflections" refers to the count of instances where the model reflects on its predictions, while "correct proportion" denotes the ratio of accurate reflections to the total reflections made.

We performed experiments on a dynamic architecture without the self-reflection mechanism. As given in Table III, removing the self-reflective mechanism from the proposed CoLLM leads to a performance decline. On FD001, the RMSE rises from 12.33 to 12.37 (a 0.3% increase), and the MAE goes from 8.86 to 8.94 (a 0.9% increase). On FD003, the RMSE increases from 11.11 to 11.16 (a 0.45% increase), and the MAE rises from 7.04 to 7.12 (a 1.1% increase). In the self-reflection samples, more than 70% of cases indicate that the SM outperforms the LM in prediction metrics, which indicates that the LM is able to correctly self-reflect and employ the SM to refine the prediction results. These findings demonstrate that the proposed self-reflection mechanism effectively enhances the accuracy of the dynamic architecture.

In addition, as shown in Fig. 4, when the confidence of the LM is low, the self-reflection mechanism leverages the decision of the smaller model for correction, mitigating unreliable predictions in complex scenarios. The upper plot presents the actual RUL values alongside the predictions generated by the SM and the LM, illustrating their respective prediction results. The lower bar chart depicts the difference in prediction errors between LM and SM [Error(LM - SM)], with red bars indicating instances where the LM's error exceeds that of the SM, and blue bars representing cases where the LM achieves lower error. A positive difference (red) suggests that the LM effectively leverages its self-reflection mechanism, utilizing the SM's predictions to further refine its own results.

D. Effect of Fuzzy Decision-Making Agent

To validate the effectiveness of the fuzzy decision-making agent, we conducted relevant experiments on the confidence

TABLE II
COMPARISON OF METHODS ON FD001 AND FD003

Methods	Training	FD001			FD003		
		RMSE	MAE	FLOPs	RMSE	MAE	FLOPs
GPT-2 [27]	1-Epoch	39.97	34.73	9.50G(0.23×)	27.42	20.62	9.50G(0.23×)
Llama2-7b [28]	1-Epoch	25.53	21.82	647.62G(0.003×)	24.32	18.07	647.26G(0.003×)
One Fits All [29]	1-Epoch	23.96	20.65	2.21G(1×)	19.91	17.39	2.21G(1×)
Autotimes [30]	1-Epoch	33.77	29.73	9.57G(0.23×)	25.06	22.35	9.57G(0.23×)
Qwen2.5-0.5b [31]	1-Epoch	35.25	31.72	35.79G(0.06×)	36.62	31.12	35.79G(0.06×)
Qwen3-0.6b [32]	1-Epoch	12.93	12.65	44.04G(0.05×)	16.31	12.02	44.04G(0.05×)
GPT-2 [27]	Fine-tuning	13.49	9.62	9.50G(0.23×)	14.20	10.48	9.50G(0.23×)
Llama2-7b [28]	Fine-tuning	14.53	10.57	647.62G(0.003×)	13.33	10.21	647.62G(0.003×)
One Fits All [29]	Fine-tuning	12.34	8.89	2.21G(1×)	11.18	7.07	2.21G(1×)
Autotimes [30]	Fine-tuning	16.70	13.73	9.57G(0.23×)	17.40	14.22	9.57G(0.23×)
Qwen2.5-0.5b [31]	Fine-tuning	13.66	9.88	35.79G(0.06×)	13.22	9.96	35.79G(0.06×)
Qwen3-0.6b [32]	Fine-tuning	13.57	9.97	44.04G(0.05×)	11.83	8.30	44.04G(0.05×)
CoLLM-A	**Multi-stage Train**	**12.45**	**9.13**	**0.57G(3.88×)**	**11.26**	**7.42**	**0.15G(14.54×)**
CoLLM-B	**Multi-stage Train**	**12.40**	**8.98**	**1.06G(2.08×)**	**11.11**	**7.23**	**0.96G(2.29×)**
CoLLM-C	**Multi-stage Train**	**12.33**	**8.86**	**1.75G(1.26×)**	**11.11**	**7.04**	**1.40G(1.57×)**

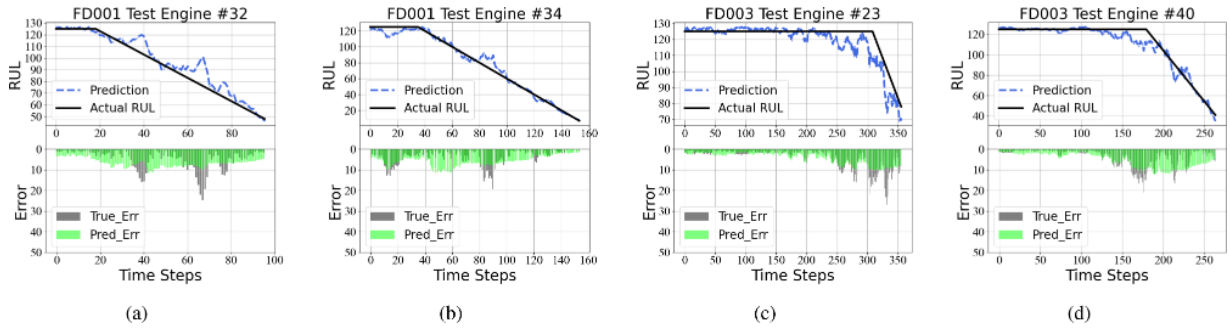


Fig. 3. Visualization of RUL prediction results. The histogram below displays the model uncertainty (green) computed by the fuzzy decision-making agent and the true prediction error (gray) of the SM. (a) FD001 #32. (b) FD001 #34. (c) FD003 #23. (d) FD003 #40.

TABLE III
EVALUATION OF THE SELF-REFLECTION MECHANISM

Model	τ_1	τ_2	Refined samples	Correct proportion	Self-Reflection	RMSE	MAE
CoLLM	0.6	0.05	249	74.2%	×	11.16	7.12
					✓	11.11	7.04
CoLLM	0.9	0.05	518	76.6%	×	11.18	7.07
					✓	11.12	6.96

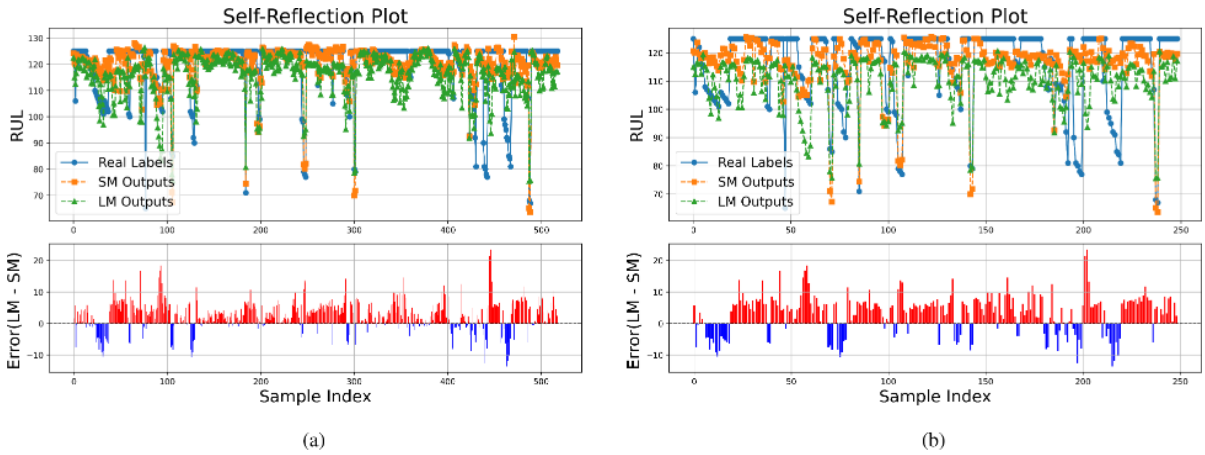
score strategy. We grouped the samples into ten confidence intervals, counted the number and percentage of samples in each interval, and then computed the RMSE within each interval. Fig. 5 illustrates the proportion of samples corresponding to different confidence levels and their respective RMSE values. We recorded the predictions from both the SM and the LM and computed the RMSE under varying confidence levels.

In particular, as shown in Fig. 5, the RMSE decreases as the confidence score increases, indicating that the confidence score effectively reflects the discrepancy between the predicted and actual RUL. This trend validates the effectiveness of the confidence prediction strategy in capturing the uncertainty of the model predictions. Furthermore, LM's higher confidence scores correspond to smaller RMSE values, suggesting that the confidence metric is meaningful.

E. Analysis of Performance Differences Between LM and SM

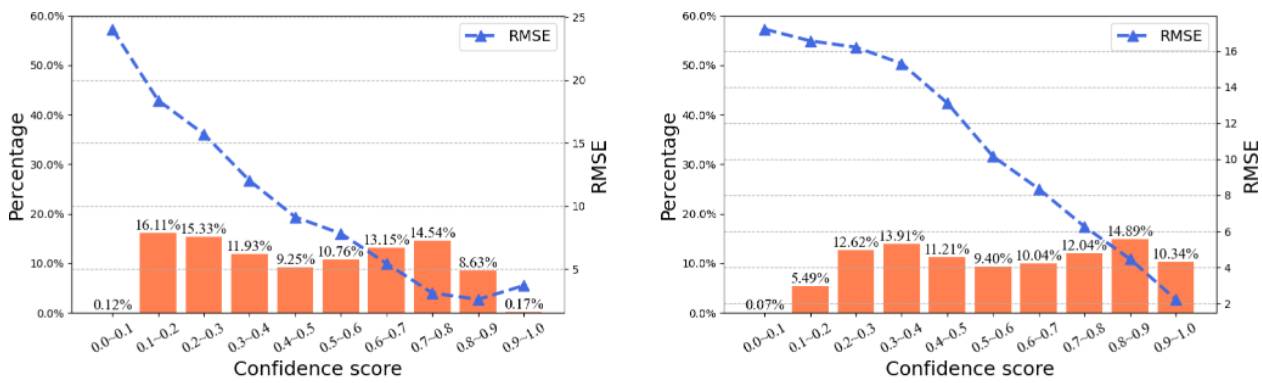
To validate that LM and SM are, respectively, suited for industrial samples with different characteristics, we conducted experiments on the RUL prediction of an engine from operational phases to fatigue failure. The prediction results are shown in Fig. 6.

As illustrated in the upper plot, the actual RUL (black line) decreases monotonically as the engine approaches failure. Both the LM (green line) and the SM (blue line) generate predictive curves that generally follow the downward trend of the actual RUL. However, there are notable differences between their performances in different stages of degradation. Taking FD001 Test Engine #34 as an example, in most of the operational phase, including the initial stage (0–40) and the later degradation period (60–160), the LM outperforms the SM by maintaining a more stable and accurate prediction trajectory. This trend is further corroborated by the lower plot in Fig. 6, where the prediction errors of the LM remain consistently lower than those of the SM for most time steps, except for the early degradation phase (40–60), where the SM exhibits lower errors. In contrast, in the early stage degradation phase (time steps 40–60), the SM exhibits better predictive precision, as its predictions align more



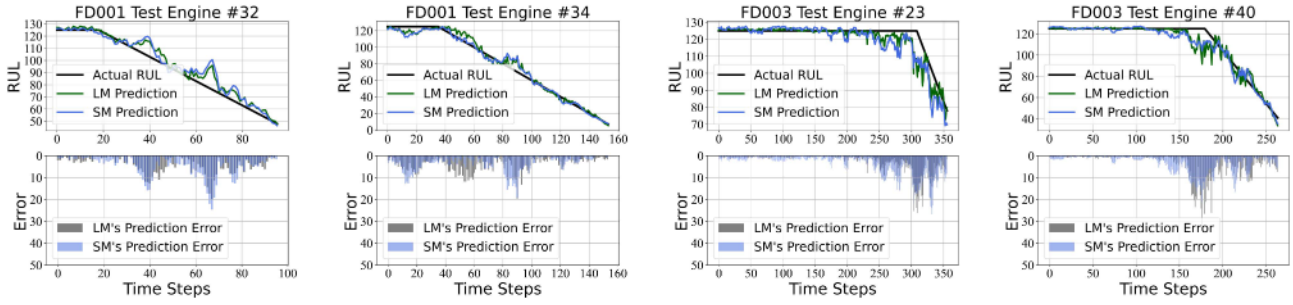
(a) (b)

Fig. 4. Visualization of self-reflection mechanism of the SM and the LM on the industrial RUL prediction task. The upper plot illustrates the true RUL values (real labels) alongside the SM and LM predictions, showing their prediction results. The lower bar chart depicts the difference in prediction errors between LM and SM [Error(LM - SM)]. Red bars indicate instances where the LM error surpasses the SM error, whereas blue bars represent cases where the LM exhibits lower error. A positive difference (red) indicates that the LM can correctly self-reflect and employ the SM to refine the prediction results. (a) Threshold setting: [0.9, 0.05]. (b) Threshold setting: [0.6, 0.05].



(a) (b)

Fig. 5. RMSE and sample distribution across confidence score intervals. The x-axis represents confidence score bins from 0.0–1.0. The left y-axis (orange bars) shows the percentage of test samples in each bin, and the right y-axis (blue dashed line) indicates the RMSE within each bin. A decreasing RMSE trend with increasing confidence score suggests that confidence estimates reflect prediction reliability. (a) SM. (b) LM.



(a) (b) (c) (d)

Fig. 6. Comparison of RUL prediction results between the LM and the SM. The upper plot illustrates the predicted RUL curves for both models, while the lower plot presents the corresponding prediction errors, where the gray bars represent the errors of the LM, and the other bars correspond to the SM. (a) FD001 #32. (b) FD001 #34. (c) FD003 #23. (d) FD003 #40

closely with the actual RUL, while the LM shows relatively larger deviations. This suggests that the SM is more effective at capturing fine-grained local variations, which are critical for early degradation detection.

These findings indicate that the strengths of LM and SM vary depending on the characteristics of the input samples. Specifically, the SM demonstrates advantages in early degradation detection by effectively capturing small fluctuations in sensor data, while the LM excels in long-term predictive stability due to its enhanced representation capacity. This performance tradeoff highlights the necessity of selecting an appropriate model based on operational conditions.

V. Conclusion

This article proposes CoLLM, a fuzzy collaborative framework that optimizes computational efficiency and reliability for industrial LMs. By constructing a fuzzy decision-making agent, the framework dynamically selects between LM and SM based on uncertainty quantification of industrial time-series data, addressing the limitations of static task-level collaboration. In addition, the self-reflection mechanism cross-validates LM outputs with SM corrections to mitigate catastrophic errors, improving decision reliability. Experiments demonstrate that CoLLM reduces computational costs of LMs by 90% while maintaining accuracy, offering a practical solution for deployment of industrial LMs.

Future work will explore integrating CoLLM with cloud–edge collaboration, where LMs are deployed in the cloud and SM at the edge to better balance performance and latency. We also aim to develop more precise uncertainty estimation methods to enhance the accuracy of large–small model collaboration.

References

[1] H. Wang, L. Ren, and Y. Li, "Metaindux-TS: Frequency-aware AIGC foundation model for industrial time series," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Jun. 23, 2025, doi: 10.1109/TNNLS.2025.3577203.

[2] L. Ren, H. Wang, and Y. Laili, "Diff-MTS: Temporal-augmented conditional diffusion-based AIGC for industrial time series toward the large model era," *IEEE Trans. Cybern.*, vol. 54, no. 12, pp. 7187–7197, Dec. 2024.

[3] M. L. Bangerter et al., "A hybrid framework integrating LLM and ANFIS for explainable fact-checking," *IEEE Trans. Fuzzy Syst.*, early access, Jul. 22, 2024, doi: 10.1109/TFUZZ.2024.3431710.

[4] L. Ren et al., "Industrial foundation model," *IEEE Trans. Cybern.*, vol. 55, no. 5, pp. 2286–2301, May 2025.

[5] A. B. Abacha et al., "MEDEC: A benchmark for medical error detection and correction in clinical notes," in *Findings Assoc. Comput. Linguistics: ACL 2025*, pp. 22539–22550.

[6] A. Liu et al., "DeepSeek-V3 technical report," 2024, *arXiv:2412.19437*.

[7] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to use large language models while reducing cost and improving performance," *Trans. Mach. Learn. Res.*, 2024, pp. 1–14.

[8] C. Zhu, X. Ma, P. D'Urso, Y. Qian, W. Ding, and J. Zhan, "Long-term multivariate time-series forecasting model based on Gaussian fuzzy information granules," *IEEE Trans. Fuzzy Syst.*, vol. 32, no. 11, pp. 6424–6438, Nov. 2024.

[9] R. Zhang, J. Zhan, W. Ding, and W. Pedrycz, "Time series forecasting based on improved multi-linear trend fuzzy information granules for convolutional neural networks," *IEEE Trans. Fuzzy Syst.*, vol. 33, no. 3, pp. 1009–1023, Mar. 2025.

[10] Z. Mei, T. Zhao, and X. Gu, "A dynamic evolving fuzzy system for streaming data prediction," *IEEE Trans. Fuzzy Syst.*, vol. 32, no. 8, pp. 4324–4337, Aug. 2024.

[11] D. Li, H. Zhang, T. Li, A. Bouras, X. Yu, and T. Wang, "Hybrid missing value imputation algorithms using fuzzy C-means and vaguely quantified rough set," *IEEE Trans. Fuzzy Syst.*, vol. 30, no. 5, pp. 1396–1408, May 2022.

[12] J. Lu, Z. Pang, M. Xiao, Y. Zhu, R. Xia, and J. Zhang, "Merge, ensemble, and cooperate! A survey on collaborative strategies in the era of large language models," 2024, *arXiv:2407.06089*.

[13] R. Prabhakar et al., "Sambanova SN40L: Scaling the AI memory wall with dataflow and composition of experts," in *Proc. 57th IEEE/ACM Int. Symp. Microarchitecture*, 2024, pp. 1353–1366.

[14] K. Lu et al., "Routing to the expert: Efficient reward-guided ensemble of large language models," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Hum. Lang. Technol. (Volume 1: Long Papers)*, 2024, pp. 1964–1974.

[15] Q. Wu, Z. Ke, Y. Zhou, X. Sun, and R. Ji, "Routing experts: Learning to route dynamic experts in multi-modal large language models," in *Proc. 12th Int. Conf. Learn. Representations*, pp. 1–15.

[16] D. Ding et al., "Hybrid LLM: Cost-efficient and quality-aware query routing," in *Proc. 12th Int. Conf. Learn. Representations*, 2024, pp. 1–19.

[17] I. Ong et al., "RouteLLM: Learning to route LLMs from preference data," in *Proc. 13th Int. Conf. Learn. Representations*, 2025, pp. 1–16.

[18] G. Ramirez, A. Birch, and I. Titov, "Optimising calls to large language models with uncertainty-based two-tier selection," in *Proc. Conf. Lang. Model.*, 2024, pp. 1–15.

[19] Z. Zhao, S. Jin, and Z. M. Mao, "Eagle: Efficient training-free router for multi-LLM inference," 2024, *arXiv:2409.15518*.

[20] G. Woo, C. Liu, A. Kumar, C. Xiong, S. Savarese, and D. Sahoo, "Unified training of universal time series forecasting transformers," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2024, pp. 53140–53164.

[21] Y. Liu, H. Zhang, C. Li, X. Huang, J. Wang, and M. Long, "Timer: Generative pre-trained transformers are large time series models," in *Proc. 41st Int. Conf. Mach. Learn.*, 2024, pp. 32369–32399.

[22] M. Jin et al., "Time-LLM: Time series forecasting by reprogramming large language models," in *Proc. Int. Conf. Learn. Representations*, 2024, pp. 1–13.

[23] Y. Bian, X. Ju, J. Li, Z. Xu, D. Cheng, and Q. Xu, "Multi-patch prediction: Adapting LLMs for time series representation learning," in *Proc. 41st Int. Conf. Mach. Learn.*, 2024, pp. 3889–3912.

[24] Z. Pan, Y. Jiang, S. Garg, A. Schneider, Y. Nevmyvaka, and D. Song, "S^2IP-LLM: Semantic space informed prompt learning with LLM for time series forecasting," in *Proc. 41st Int. Conf. Mach. Learn.*, 2024, pp. 39135–39153.

[25] A. Saxena, K. Goebel, D. Simon, and N. Eklund, "Damage propagation modeling for aircraft engine run-to-failure simulation," in *Proc. Int. Conf. Prognostics Health Manage.*, 2008, pp. 1–9.

[26] L. Ren, H. Wang, and G. Huang, "DLformer: A dynamic length transformer-based network for efficient feature representation in remaining useful life prediction," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 5, pp. 5942–5952, May 2024.

[27] A. Radford et al., "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, 2019, Art. no. 9.

[28] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288*.

[29] T. Zhou et al., "One fits all: Power general time series analysis by pre-trained LM," *Adv. Neural Inf. Process. Syst.*, vol. 36, pp. 43322–43355, 2023.

[30] Y. Liu, G. Qin, X. Huang, J. Wang, and M. Long, "AutoTimes: Autoregressive time series forecasters via large language models," *Adv. Neural Inf. Process. Syst.*, vol. 37, pp. 122154–122184, 2024.

[31] A. Yang et al., "QWEN2. 5 technical report," 2024, *arXiv:2409.12186*.

[32] A. Yang et al., "QWEN3 technical report," 2025, *arXiv:2505.09388*.

[33] L. Ren, H. Wang, Y. Wang, K. Huang, L. Wang, and B. Li, "Foundation models for the process industry: Challenges and opportunities," *Engineering*, 2025, doi: 10.1016/j.eng.2025.03.023.

[34] L. Ren, J. Li, and H. Wang, "An AIGC-driven score-based diffusion approach for industrial time series," *IEEE Internet Things J.*, vol. 12, no. 10, pp. 13132–13143, 2025.

[35] S. Xie et al., "Hierarchical relationship modeling in multi-agent reinforcement learning for mixed cooperative–competitive environments," *Inf. Fusion*, vol. 108, Art. no. 102318, 2024.

[36] S. Xie, H. Zhang, H. Yu, Y. Li, Z. Zhang, and X. Luo, "ET-HF: A novel information sharing model to improve multi-agent cooperation," *Knowl.-Based Syst.*, vol. 257, 2022, Art. no. 109916.

[37] Z. Zhang, S. Xie, H. Zhang, X. Luo, and H. Yu, "Zero-shot sim-to-real transfer using siamese-Q-based reinforcement learning," *Inf. Fusion*, vol. 114, 2025, Art. no. 102664.

Haiteng Wang (Member, IEEE) received the Ph.D. degree in pattern recognition and intelligent systems from the School of Automation Science and Electrical Engineering, Beihang University, Beijing, China, in 2026.

He is currently a Post-Doctoral Fellow with Beihang University. His current research interests include time series prediction, industrial foundation models, and generative models.

Lei Ren (Senior Member, IEEE) received the Ph.D. degree in computer science from the Institute of Software, Chinese Academy of Sciences, Beijing, China, in 2009.

He is currently a Professor with the School of Automation Science and Electrical Engineering, Beihang University, Beijing, with the Hangzhou International Innovation Institute, Beihang University, Hangzhou, China, and also with the State Key Laboratory of Intelligent Manufacturing System Technology, Beijing. His research interests include neural networks and deep learning, time series analysis, and industrial AI applications.

Dr. Ren serves as an Associate Editor for IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS, IEEE/ASME TRANSACTION ON MECHATRONICS, and other international journals.

Tuo Zhao received the B.S. degree in automation from the College of Automation Engineering, Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2023. He is currently working toward the master's degree in control engineering with the School of Automation Science and Electrical Engineering, Beihang University, Beijing.

His research interests include time series prediction and dynamic neural networks.

Lu Jiao received the B.Eng. degree in artificial intelligence from the Beijing University of Chemical Technology, Beijing, China, in 2024. She is currently working toward the M.S. degree in control engineering with the School of Automation Science and Electrical Engineering, Beihang University, Beijing, China.

Her current research interests include time series prediction, industrial large-scale models, and their applications in smart manufacturing.